



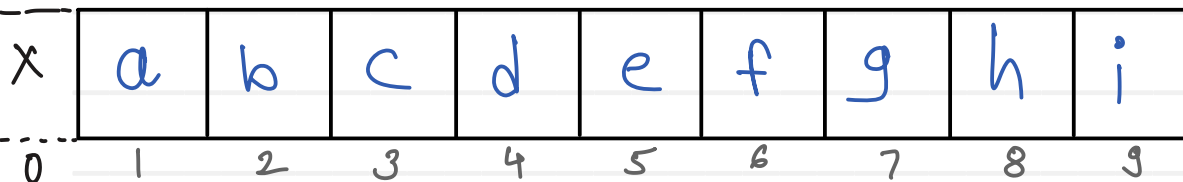
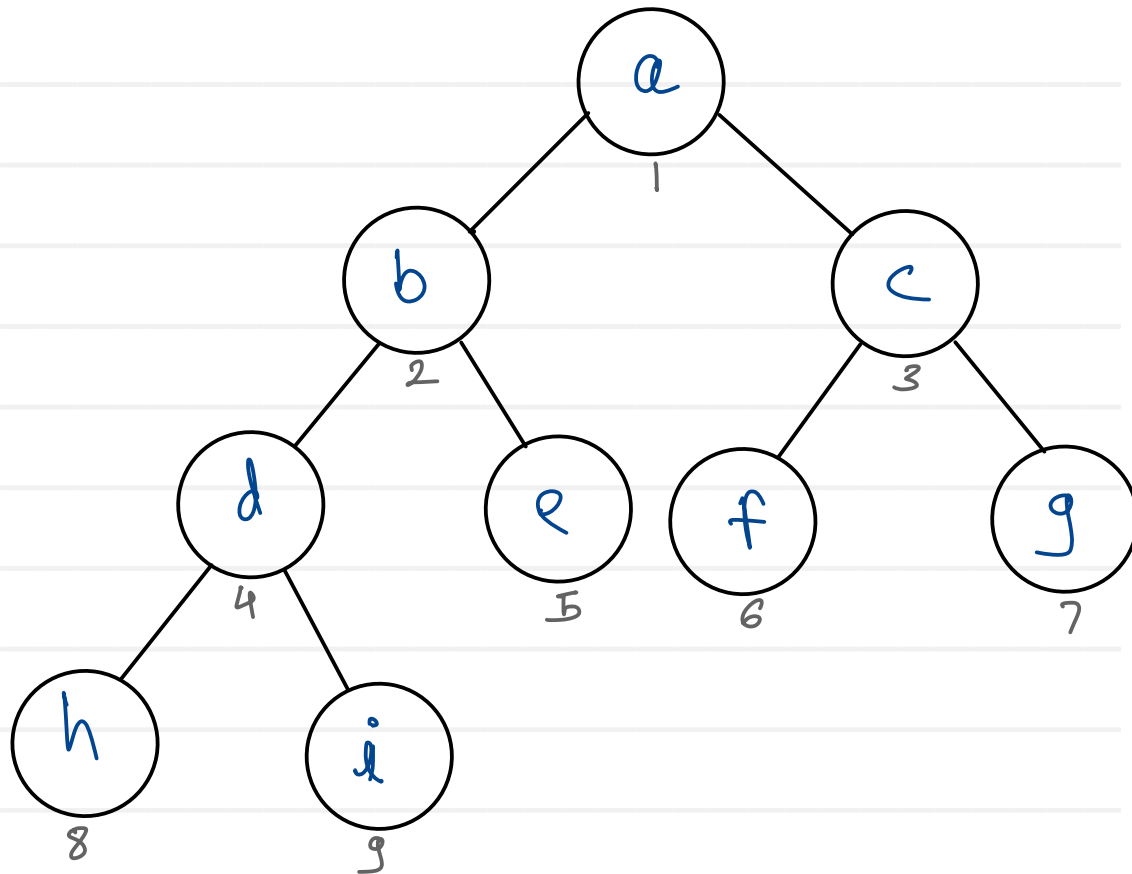
Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Complete Binary Tree or Heap



- Complete Binary Tree (height = h)
- All levels should be completely filled except last
- All leaf nodes must be at level h or h-1
- All leaf nodes at level h must aligned as left as possible
- Array implementation of Complete Binary Tree is called as heap

- array indices are used to maintain parent child relationship

node - i^{th} index

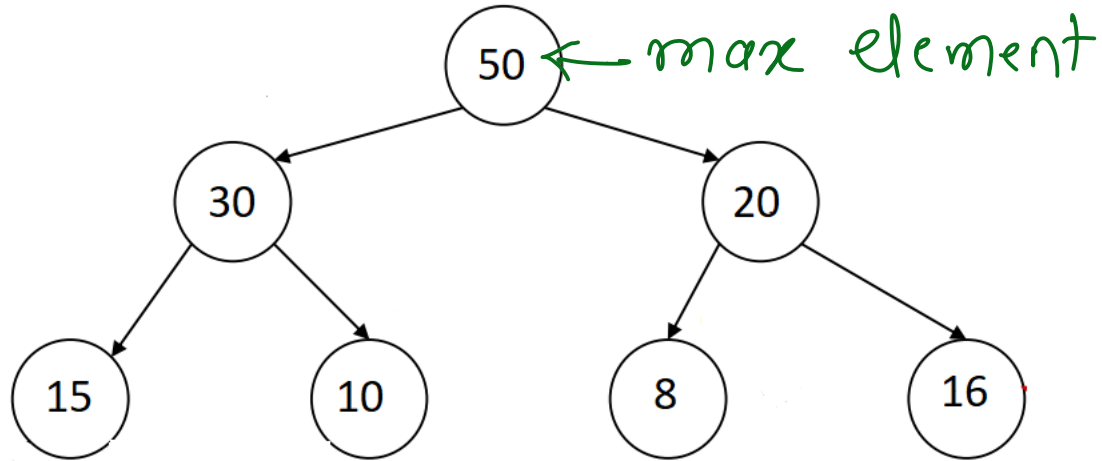
parent - $i/2$ index

left child - $i * 2$ index

right child - $i * 2 + 1$ index

Heap Types – Max and Min

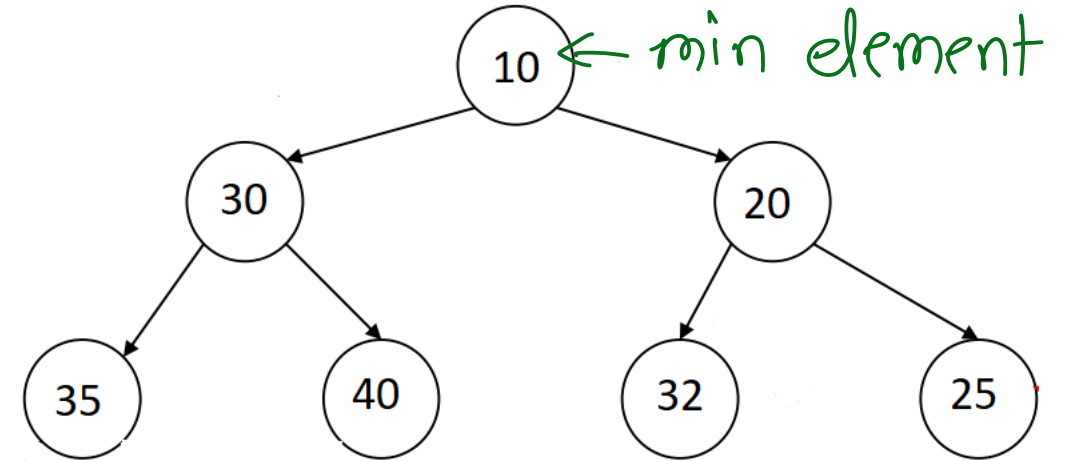
Max Heap



50	30	20	15	10	8	16
1	2	3	4	5	6	7

- Max heap is a heap data structure in which each node is greater than both of its child nodes.

Min Heap

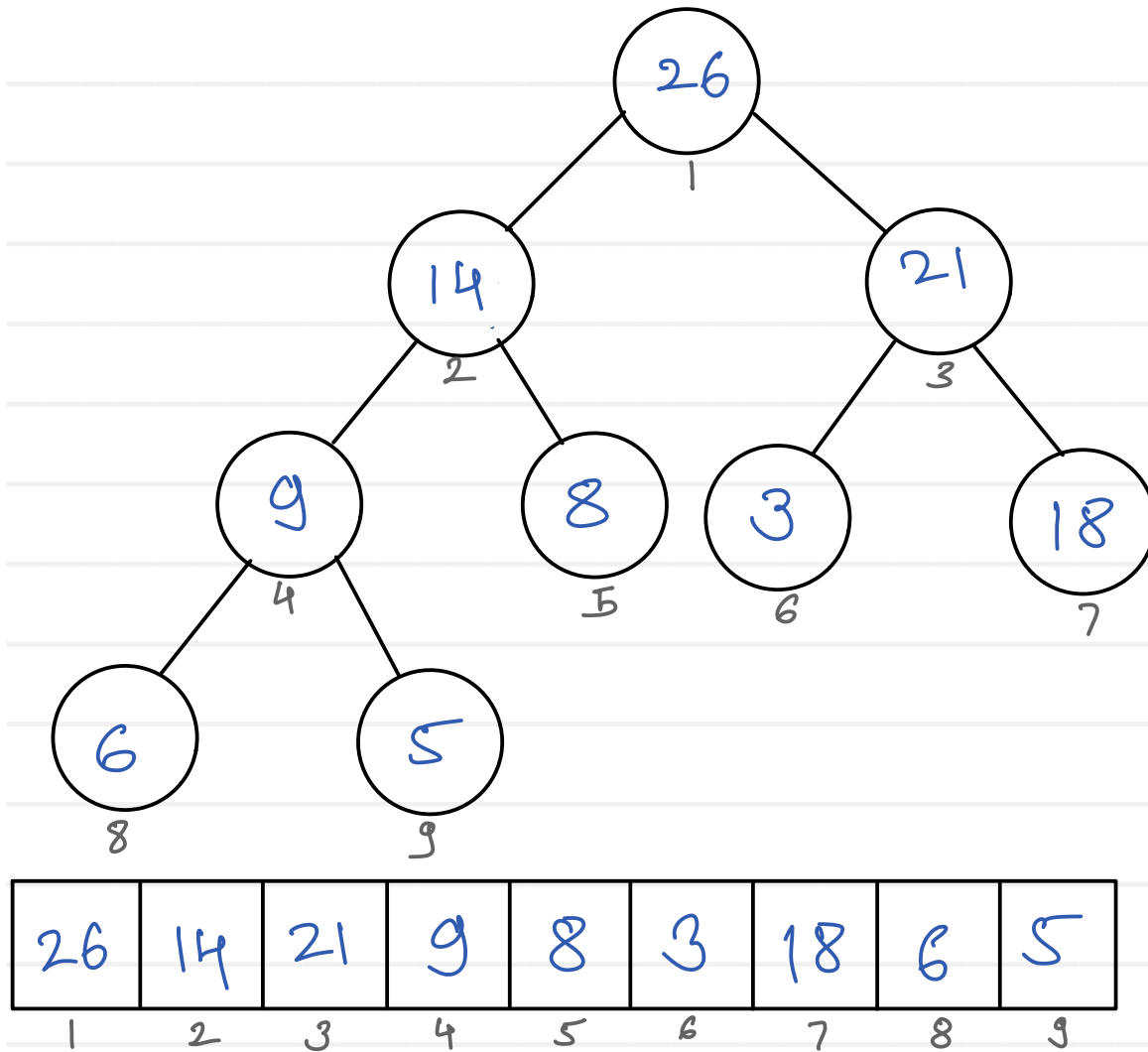


10	30	20	35	40	32	25
1	2	3	4	5	6	7

- Min heap is a heap data structure in which each node is smaller than both of its child nodes.



Heap - Create heap (Add)



Keys : 6, 14, 3, 26, 8, 18, 21, 9, 5

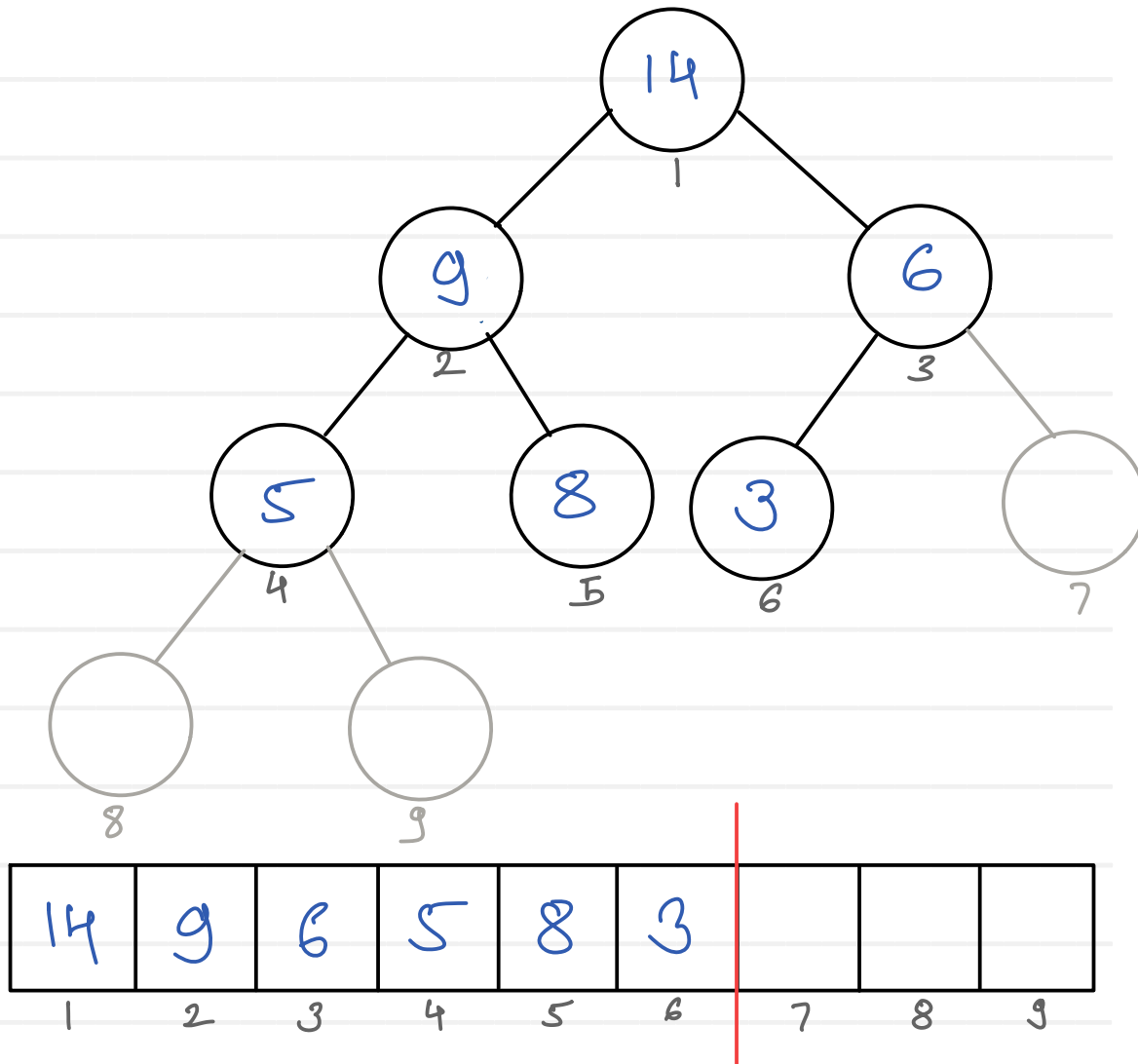
Algorithms:

1. Add new value on first empty index from left side.
2. Adjust position of new value by comparing it with its ancestors one by one.

- to add new value, may need to traverse from leaf to root

$$T(n) = O(\log n)$$

Heap - Delete heap (Delete)



Property : can delete only root node from heap

1. in max heap, always maximum element will be deleted from heap.
2. in min heap, always minimum element will be deleted from heap.

max value = 26, 21, 18

Algorithm:

1. place last value of heap on root place.
2. Adjust position of root element by comparing with all its descendants one by one

- to adjust position, may need to traverse from root to leaf

$$T(n) = O(\log n)$$

p_i	c_i	p_i			c_i	p_i		
21	14	18	9	8	3	5	6	
1	2	3	4	5	6	7	8	9

```

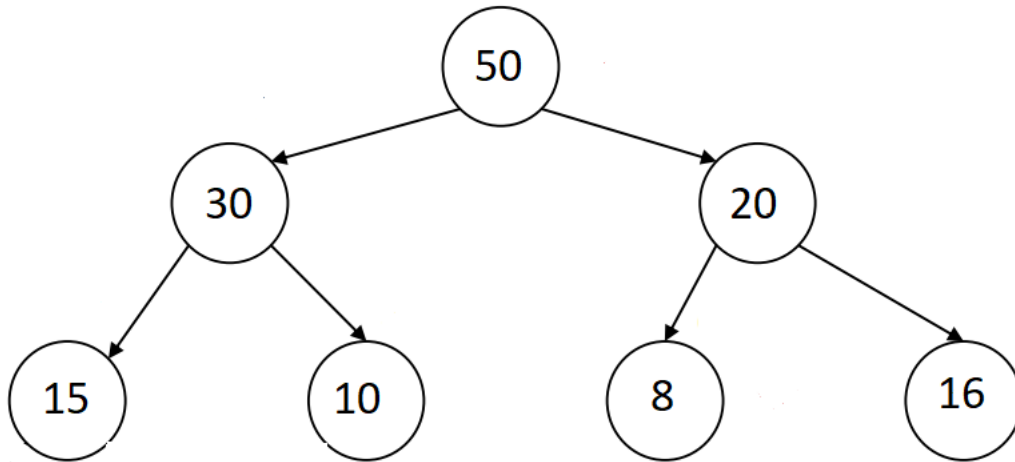
 $p_i = 1;$ 
 $c_i = p_i * 2;$ 
while( $c_i \leq size$ ) {
    |
    |
    |
    |
     $p_i = c_i;$ 
     $c_i = p_i * 2;$ 
}

```

$Max = 26, size = 8$
 c_i p_i c_i $c_i + 1$
 1 2 3
 3 6 7

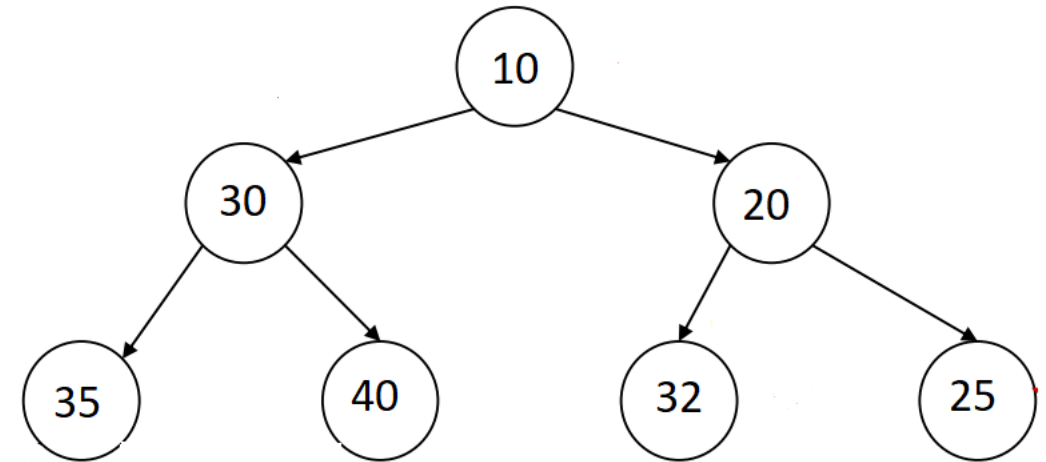
Priority Queues

**Higher number
Higher priority**



- In Max heap always root element which has highest value is removed

**Lower number
Higher priority**



- In Min heap always root element which has lowest value is removed



Priority Queue

- Always high priority element is deleted from queue
- value (priority) is assigned to each element of queue
- priority queue can be implemented using array or linked list.
- to search high priority data (element) need to traverse array or linked list
- Time complexity = $O(n)$

- priority queue can also be implemented using heap.
because, maximum/minimum value is kept at root position in max heap & min heap respectively.
- push, pop & peek will be performed efficiently

max value $\rightarrow$ high priority $\rightarrow$ max heap
min value $\rightarrow$ high priority $\rightarrow$ min heap

- Queue where high priority data is always peeked or deleted.
- Priority can be implemented using array, linked list and heap data structures.
- An array is used to store a value and its associated priority. In some simpler implementations, the value itself might represent the priority (e.g., lower value means higher priority).
- Array and linked list implementation of priority queue often leads to less efficient performance compared to heap-based implementations for insertion and deletion operations.

- **Ordered vs. Unordered Array**

- **Ordered Array / *linked list***

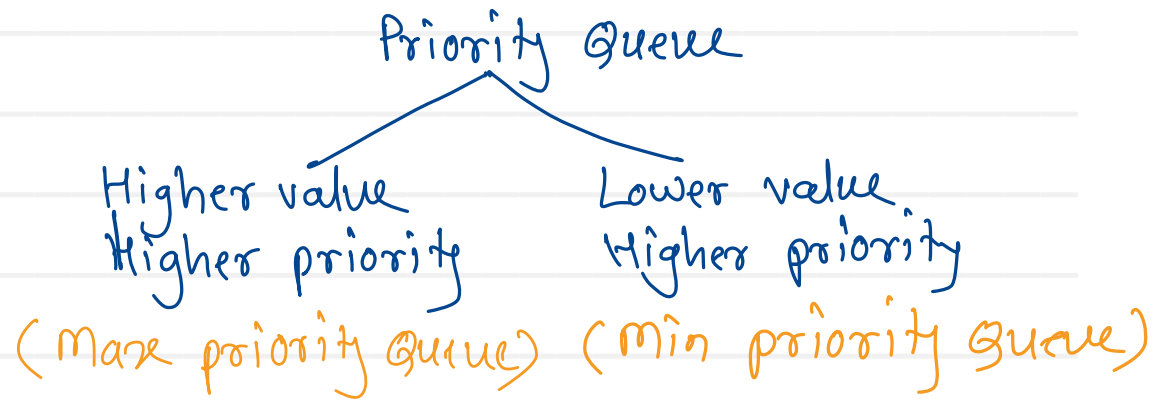
- Elements are kept sorted by priority. Insertion requires shifting existing elements to maintain order, leading to $O(n)$ time complexity for insertion.
 - Deletion of the highest priority element is $O(1)$ as it's typically at the beginning or end of the array.

- **Unordered Array: *linked list***

- Elements are inserted without regard to order, making insertion $O(1)$.
 - Deletion of the highest priority element requires searching the entire array to find it, resulting in $O(n)$ time complexity for deletion.

Priority Queue Implementation

- Priority : number associated with value
- Priority range is defined by programmer
e.g. Priority range : 1 to 10



- Every element of priority queue will have two parts:

value : any data type
priority : integer

```
struct item {
    int value;
    int priority;
};
```

```
struct priorityQueue {
    struct item arr[5];
    int capacity;    (maxSize)
    int size;
};
```

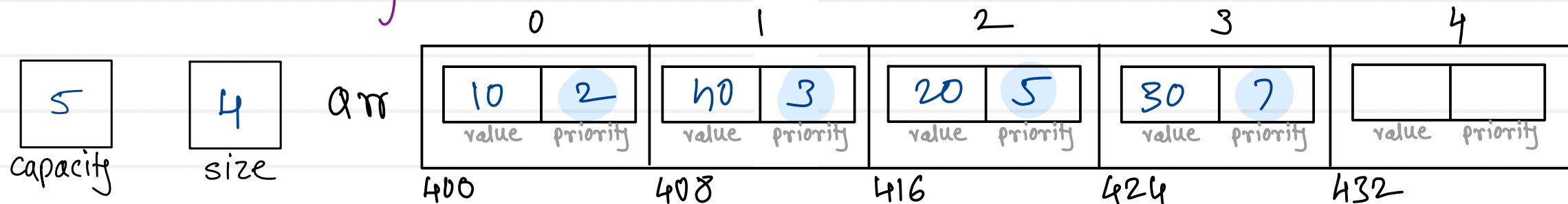
Operations :

- 1) Enqueue
- 2) Dequeue
- 3) Peek

4) isEmpty $\rightarrow$ size == 0
5) isFull $\rightarrow$ size == capacity

Priority Queue Implementation

Ordered array:



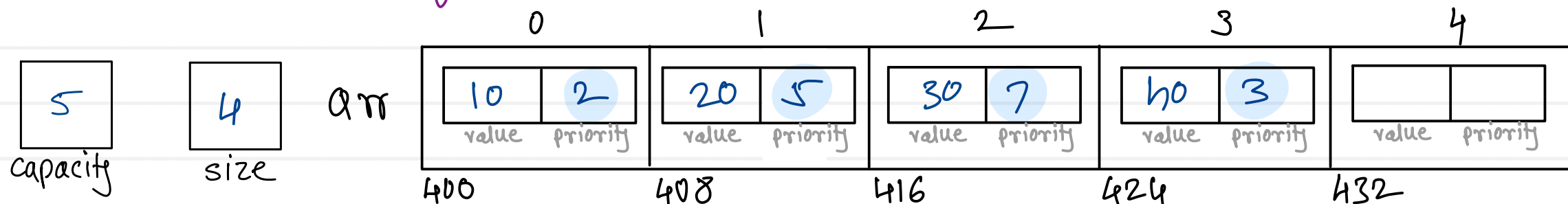
insert - $O(n)$

delete - $O(1)$

peek - $O(1)$

for shifting - $O(n)$

Unordered array:



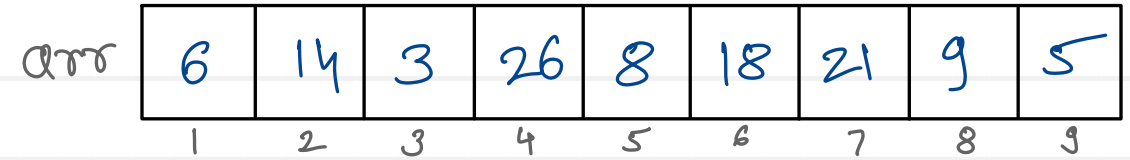
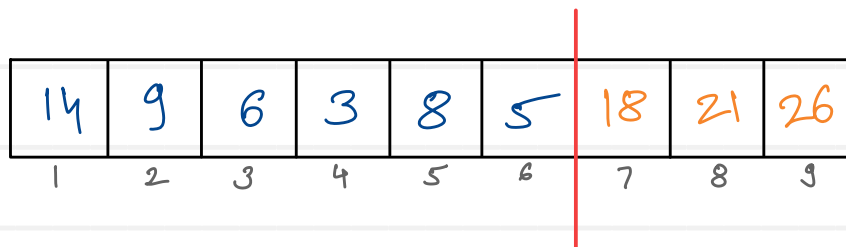
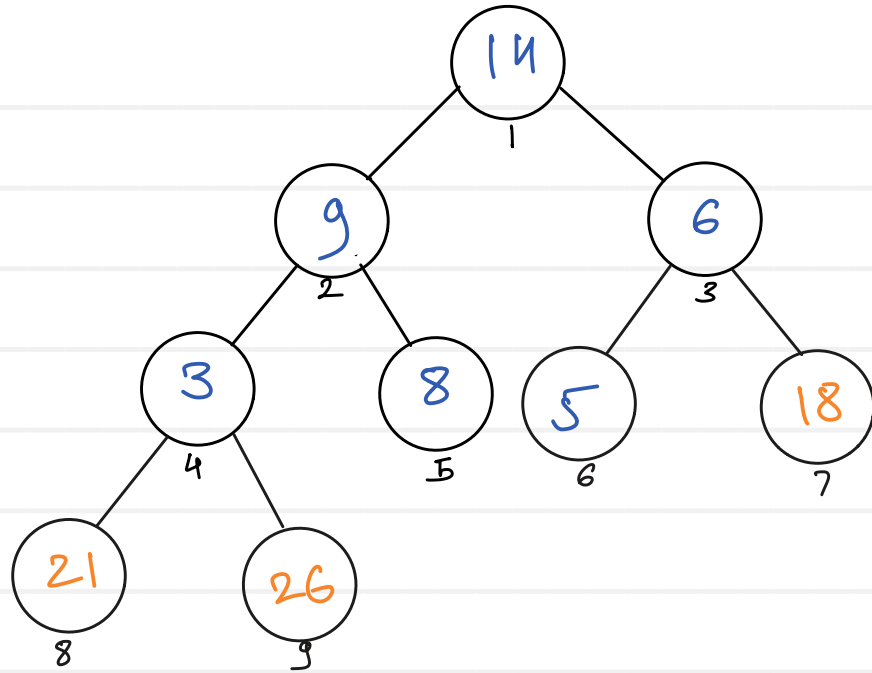
insert - $O(1)$

delete - $O(n)$

peek - $O(n)$

for shifting - $O(n)$

Heap sort



Algorithm :

1. convert given array into either max heap / min heap

↓ ↓
 Ascending Descending
 orders orders

2. Delete all elements from heap one by one & keep them on vacant places from right side into array.

to create heap : $n \log n$

to delete heap : $n \log n$

$$T \propto \frac{n \log n}{2}$$

heapify = n

$$S(n) = O(1)$$

$$T(n) = O(n \log n)$$

Best
Avg
Worst

Graph : Terminologies

- **Graph** is a non linear data structure having set of vertices (nodes) and set of edges (arcs).

- $G = \{V, E\}$

Where V is a set of vertices and E is a set of edges

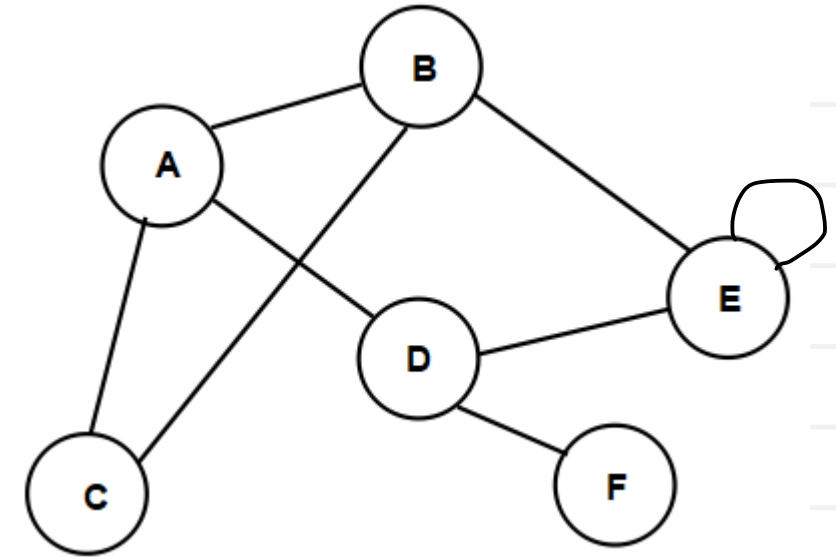
- **Vertex (node)** is an element in the graph

$$V = \{A, B, C, D, E, F\}$$

- **Edge (arc)** is a line connecting two vertices

$$E = \{(A,B), (A,C), (B,C), (B,E), (D, E), (D,F), (A,D)\}$$

- Vertex A is set be adjacent to B, if and only if there is an edge from A to B.
- **Degree of vertex** :- Number of vertices adjacent to given vertex
- **Path** :- Set of edges connecting any two vertices is called as path between those two vertices.
 - Path between A to D = $\{(A, B), (B, E), (E, D)\}$
- **Cycle** :- Set of edges connecting to a node itself is called as cycle.
 - $\{(A, B), (B, E), (E, D), (D, A)\}$
- **Loop** :- An edge connecting a node to itself is called as loop. Loop is smallest cycle.



- **Undirected graph.**

- If we can represent any edge either (u,v) OR (v,u) then it is referred as **unordered pair of vertices** i.e. **undirected edge**.
- **graph which contains undirected edges referred as undirected graph.**



$$(u, v) == (v, u)$$

- **Directed Graph (Di-graph)**

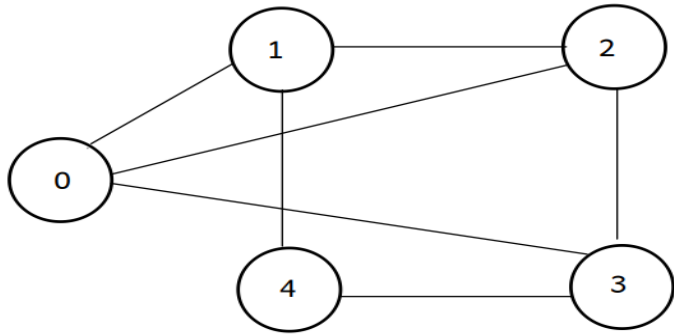
- If we cannot represent any edge either (u,v) OR (v,u) then it is referred as an **ordered pair of vertices** i.e. **directed edge**.
- **graph which contains set of directed edges referred as directed graph (di-graph).**
- **graph in which each edge has some direction**



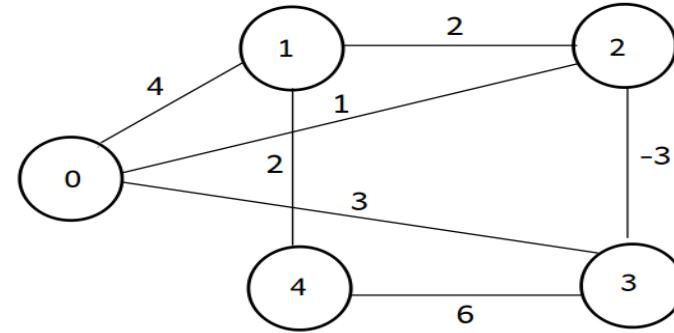
$$(u, v) \neq (v, u)$$

- **Weighted Graph**

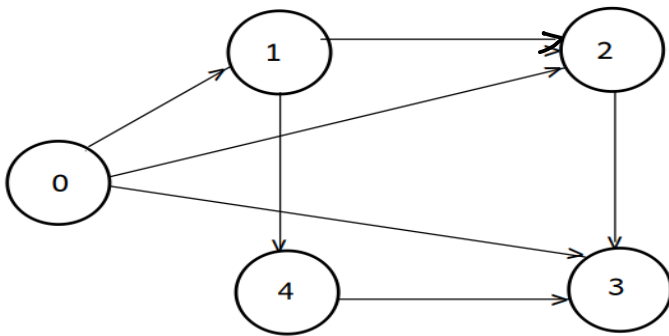
- A graph in which edge is associated with a number (ie weight)



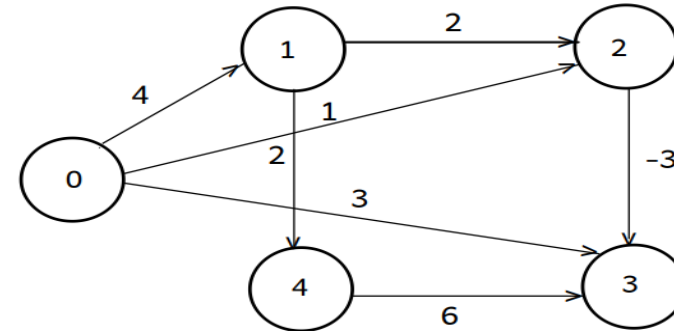
undirected unweighted graph



undirected weighted graph



directed unweighted graph



directed weighted graph

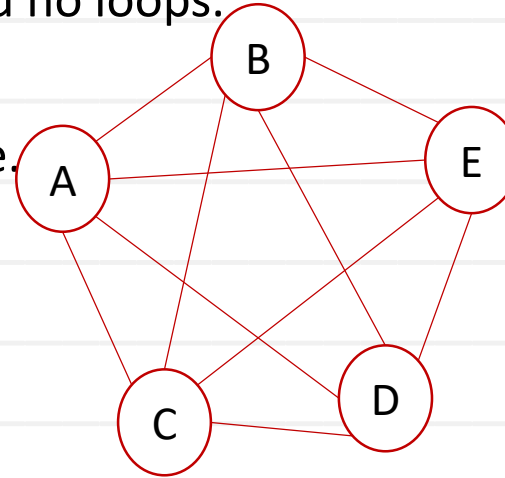
Graph : Types

- **Simple Graph**

- Graph not having multiple edges between adjacent nodes and no loops.

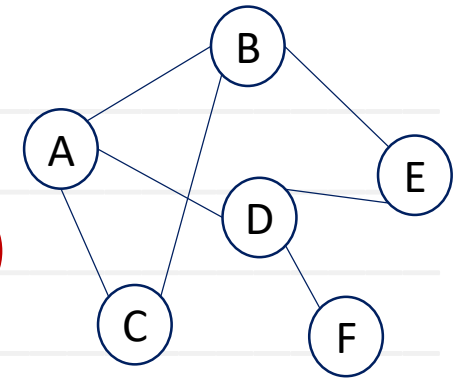
- **Complete Graph**

- Simple graph in which node is adjacent with every other node.
- Un-Directed graph: Number of Edges = $n(n-1)/2$
where, n – number of vertices
- Directed graph: Number of edges = $n(n-1)$



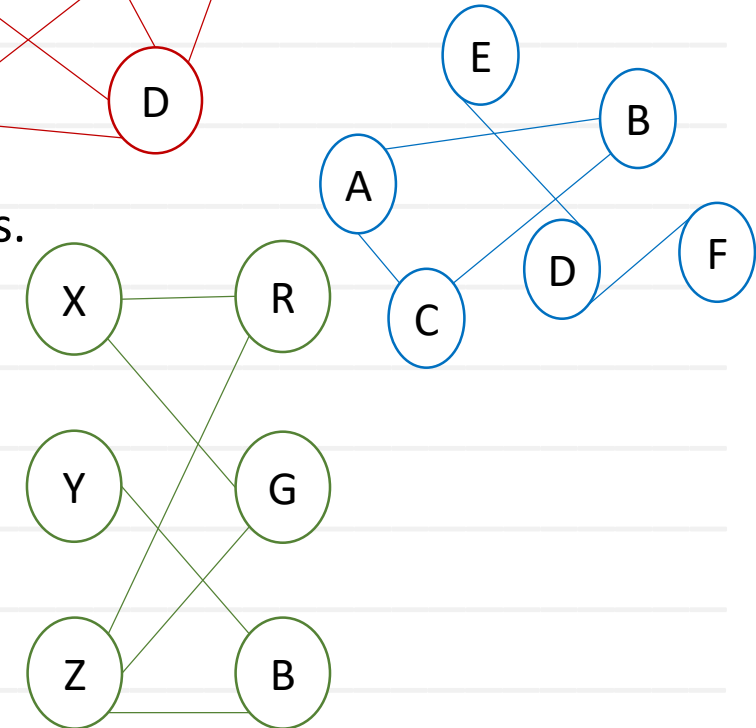
- **Connected Graph**

- Simple graph in which there is some path exist between any two vertices.
- Can traverse the entire graph starting from any vertex.



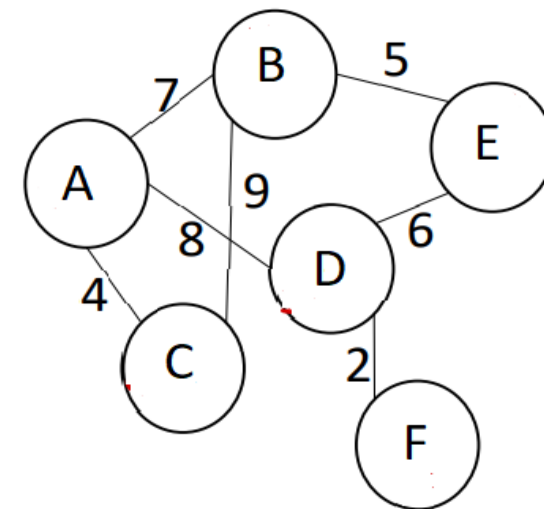
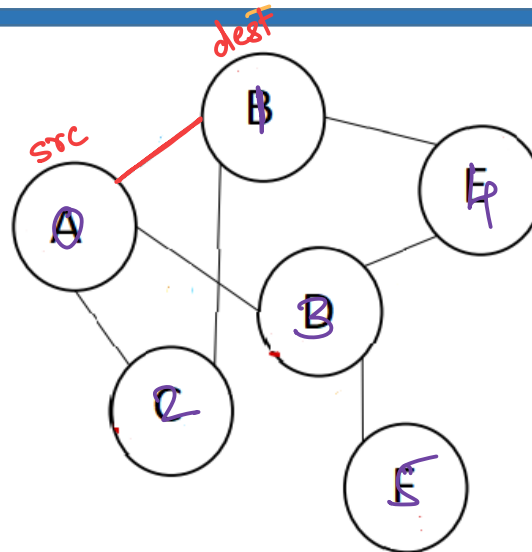
- **Bi-partite graph**

- Vertices can be divided in two disjoint sets.
- Vertices in first set are connected to vertices in second set.
- Vertices in a set are not directly connected to each other.



Graph Implementation – Adjacency Matrix

- If graph have V vertices, a V x V matrix can be formed to store edges of the graph.
- Each matrix element represent presence or absence of the edge between vertices.
- For non-weighted graph, 1 indicate edge and 0 indicate no edge.
- For weighted graph, weight value indicate the edge and infinity sign ∞ represent no edge.
- For un-directed graph, adjacency matrix is always symmetric across the diagonal.
- Space complexity of this implementation is $O(V^2)$.

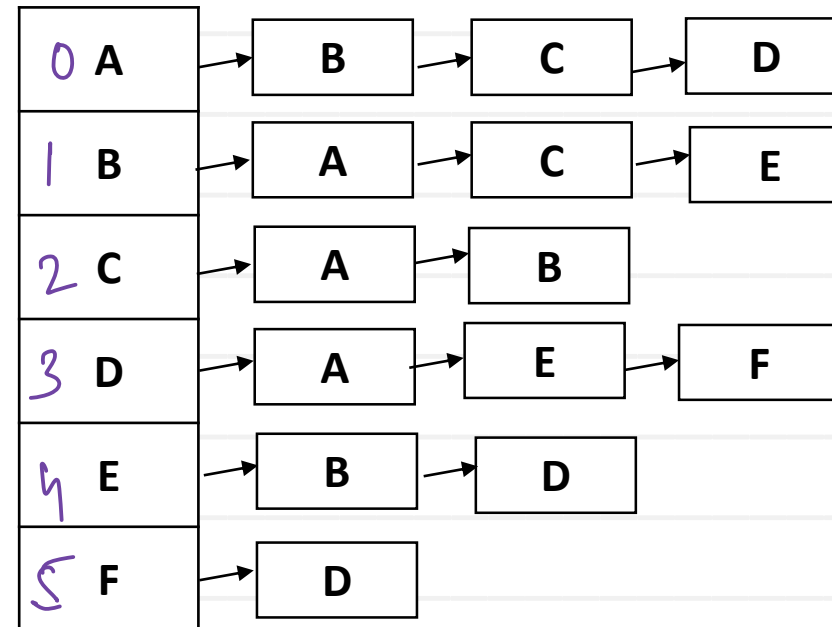
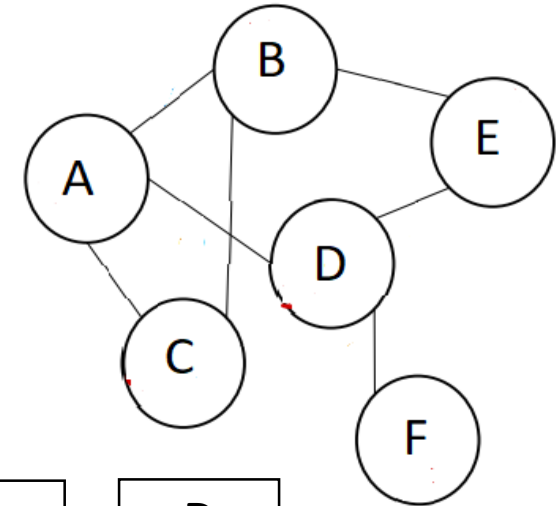


	A	B	C	D	E	F
A	0	1	1	1	0	0
B	1	0	1	0	1	0
C	1	1	0	0	0	0
D	1	0	0	0	1	1
E	0	1	0	1	0	0
F	0	0	0	1	0	0

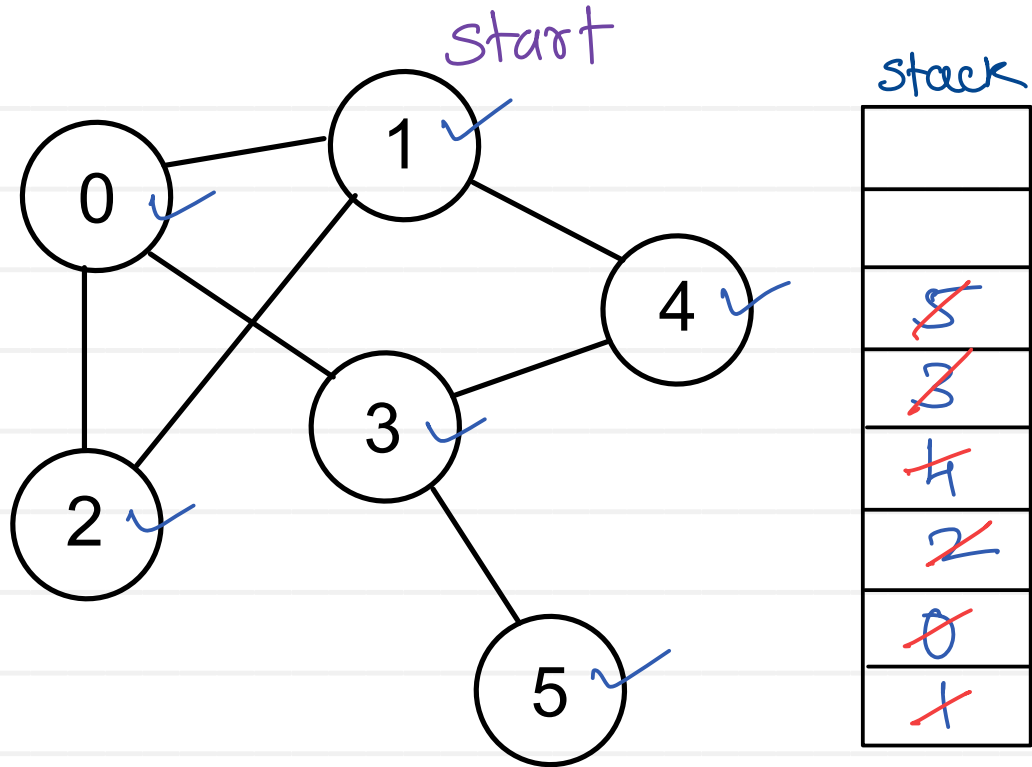
	A	B	C	D	E	F
A	∞	7	4	8	∞	∞
B	7	∞	9	∞	5	∞
C	4	9	∞	∞	∞	∞
D	8	∞	∞	∞	6	2
E	∞	5	∞	6	∞	∞
F	∞	∞	∞	2	∞	∞

Graph Implementation – Adjacency List

- Each vertex holds list of its adjacent vertices.
- For non-weighted graphs only, neighbor vertices are stored.
- For weighted graph, neighbor vertices and weights of connecting edges are stored.
- Space complexity of this implementation is $O(V+E)$.
- If graph is sparse graph (with fewer number of edges), this implementation is more efficient (as compared to adjacency matrix method).

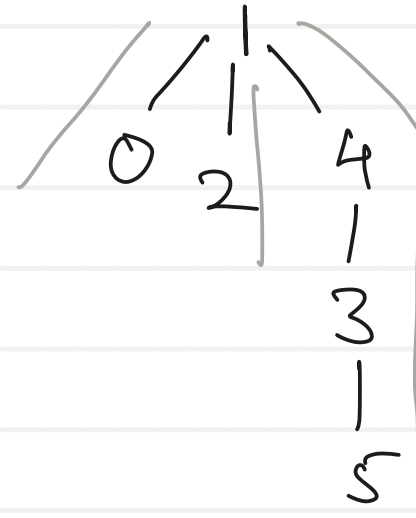


DFS Traversal

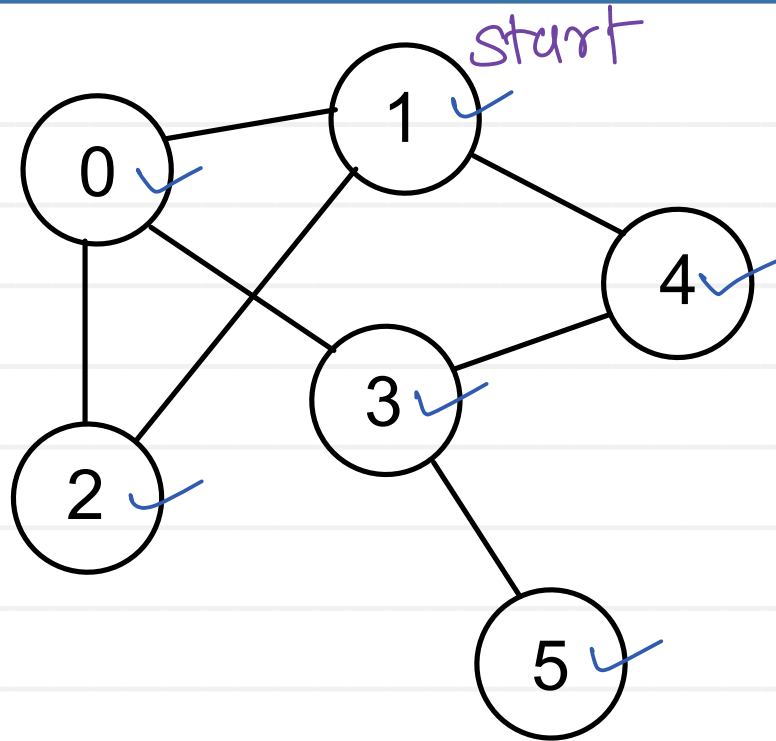


Traversal: 1, 4, 3, 5, 2, 0

1. Choose a vertex as start vertex.
2. Push start vertex on stack & mark it.
3. Pop vertex from stack.
4. Print the vertex.
5. Put all non-visited neighbours of the vertex on the stack and mark them.
6. Repeat 3-5 until stack is empty.



BFS Traversal



Traversal : 1, 0, 2, 4, 3, 5

1. Choose a vertex as start vertex.
2. Push start vertex on queue & mark it
3. Pop vertex from queue.
4. Print the vertex.
5. Put all non-visited neighbours of the vertex on the queue and mark them.
6. Repeat 3-5 until queue is empty.

